

Shell Showdown

Bash, Zsh & Fish: Eficiencia en la Terminal

Tutorial para 42 Málaga

01. Introducción

Bash vs Zsh vs Fish

Ecosistema de Shells

Análisis comparativo de características

Bash

- AÑO DE LANZAMIENTO
1989
- COMPATIBILIDAD POSIX
100% Estándar
- FILOSOFÍA DE SINTAXIS
Clásica y estricta
- EXPERIENCIA INICIAL
Espartana

Plugins: Limitados

Zsh

- AÑO DE LANZAMIENTO
1990
- COMPATIBILIDAD POSIX
99% (Superconjunto)
- FILOSOFÍA DE SINTAXIS
Clásica mejorada
- EXPERIENCIA INICIAL
Espartana

Plugins: Oh My Zsh (Extenso)

Fish Shell

- AÑO DE LANZAMIENTO
- COMPATIBILIDAD POSIX
- FILOSOFÍA DE SINTAXIS
- EXPERIENCIA INICIAL

Plugins: Fisher (Ligero/Nativo)

Ventajas



Bash

- ✓ **Portabilidad absoluta:**
Linux, macOS, WSL, servidores, contenedores.
- ✓ **Documentación infinita:**
todo problema ya fue resuelto en internet.
- ✓ **Estándar:**
El rey en CI/CD, Dockerfiles y scripts de instalación.
- ✓ **Ligero:**
Sin dependencias externas y ultra rápido en arranque.
- ✓ **Compatibilidad total:**
Scripts de hace décadas siguen funcionando.



Zsh

- ✓ **Compatibilidad total:**
Los scripts de Bash funcionan sin cambios.
- ✓ **Oh My Zsh:**
El mayor ecosistema de plugins y temas de la comunidad.
- ✓ **Personalizable:**
Puedes redefinir cualquier comportamiento interno.
- ✓ **Nativo en Apple:**
Es la Shell por defecto en macOS actualmente.
- ✓ **Mejor autocompletado:**
Supera a Bash ampliamente.
- ✓ **Arrays flexibles:**
Manipulación limpia sin requerir `${array[@]}`.



Fish

- ✓ **Autocompletado nativo:**
Lee páginas `man` y sugiere comandos en gris.
- ✓ **Sintaxis limpia:**
Adiós a `do/done/fi`, todo termina con `end`.
- ✓ **argparse integrado:**
Procesa banderas avanzadas en una sola línea.
- ✓ **Listas nativas:**
Manejo de arrays real sin comillas de protección.
- ✓ **funcsave :**
Guardado instantáneo de funciones desde la terminal.
- ✓ **Rápido y robusto:**
Diseñado desde cero y escrito en C++.

Desventajas



Bash

- **✗ Sintaxis arcaica:**

Un espacio en blanco mal puesto rompe el script por completo.

- **✗ Espacios en rutas:**

Olvidar las comillas dobles desarma los argumentos.

- **✗ Parseo tedioso:**

Requiere bucles `while + case + shift` para banderas simples.

- **✗ Autocompletado básico:**

Por defecto viene limitado a comandos y archivos locales.



Zsh

- **✗ Requiere configuración:**

Sin plugins externos, es visualmente tan gris como Bash.

- **✗ Lentitud:**

Cargar Oh My Zsh con muchos temas puede ralentizar el arranque.

- **✗ Sintaxis heredada:**

Mantiene la lógica pesada de usar `fi`, `done` y `[[ ]]`.

- **✗ Inconsistencias:**

Algunas funciones nativas de Bash no se comportan igual aquí.

- **✗ Dependencias pesadas:**

Oh My Zsh puede convertirse en un monstruo difícil de auditar.



Fish

- **✗ Incompatibilidad POSIX:**

Los scripts de Bash tradicionales no funcionarán nativamente.

- **✗ Ausencia en servidores:**

Rara vez viene instalado por defecto en servidores de producción.

- **✗ Ecosistema menor:**

Tiene menos recursos listos para usar que Oh My Zsh.

- **✗ Herramientas CLI externas:**

A veces requieren envoltorios como `bass` o traducción manual.

- **✗ Curva de aprendizaje:**

Su sintaxis diferente puede chocar a usuarios veteranos de Bash/Zsh.

Variables y Condicionales

Bash/Zsh

```
1 nombre="42Malaga"
2
3 if [ -d "$HOME/Downloads" ]; then
4     echo "$nombre: carpeta existe"
5 fi
```

Reglas estrictas:

- Sin espacios alrededor del `=`
- Espacios **obligatorios** dentro de `[ ]`
- Las comillas dobles protegen espacios

Fish

```
1 set nombre "42Malaga"
2 set -l ruta_local "$HOME/Downloads"
3 set -gx VARIABLE_GLOBAL "exportada"
4
5 if test -d $HOME/Downloads
6     echo "Carpeta existe"
7 end
```

Reglas más flexibles:

- `set` reemplaza al `=` — sin errores de espacios
- `test` sin corchetes, cierra con `end`
- Variables globales con `set -gx`
- Variables locales con `set -l`

Arrays y Bucles

Bash/Zsh

```
1 extensiones=(js py ts)
2
3 for ext in "${extensiones[@]"; do
4     echo "Procesando: $ext"
5 done
```

Sintaxis pesada: `${array[@]}` , `do` / `done`

Fish

```
1 set extensiones js py ts
2
3 for ext in $extensiones
4     echo "Procesando: $ext"
5 end
```

- Las listas son nativas: `$extensiones` ya es una lista
- Sin `${array[@]}` , sin `do` / `done`

Brace Expansion & Argumentos

Bash/Zsh

```
1  # Crear carpetas al vuelo
2  mkdir test{1..4}
3  mkdir -p proyecto/{js,css}/{src,dist}
4
5  # Parseo manual de argumentos
6  ejemplo_args() {
7      local output=""
8      while [[ $# -gt 0 ]]; do
9          case "$1" in
10             -o|--output) output="$2"; shift 2 ;;
11             *) echo "Error: $1"; return 1 ;;
12          esac
13      done
14  }
```

Fish

```
1  # Brace expansion (desde Fish 3.0)
2  mkdir test{1..4}
3  mkdir -p proyecto/{js,css}/{src,dist}
4
5  # Parseo de argumentos en una línea
6  function ejemplo_args
7      argparse 'o/output=' 'e/ext=+' -- $argv
8      echo "Salida: $_flag_output"
9      echo "Extensiones: $_flag_ext"
10 end
11
```

05. Comparativa Directa

Variables

CONCEPTO	BASH / ZSH	FISH
Declarar	<code>var="x"</code>	<code>set var "x"</code>
Local	<code>local var="x"</code>	<code>set -l var "x"</code>
Global	<code>export VAR=x</code>	<code>set -gx VAR x</code>
Espacios en <code>=</code>	✗ Prohibido	✓ No aplica

Condicionales

CONCEPTO	BASH / ZSH	FISH
If	<code>if ["\$a" = "b"]</code>	<code>if test \$a = "b"</code>
And	<code>&& en []</code>	<code>; and</code>
Or	<code> en []</code>	<code>; or</code>
Cierre	<code>fi</code>	<code>end</code>

Arrays / Listas

CONCEPTO	BASH / ZSH	FISH
Declarar	<code>arr=(a b)</code>	<code>set arr a b</code>
Acceder	<code>"\${arr[0]}"</code> (Base 0)	<code>\$arr[1]</code> (Base 1)
Todos	<code>"\${arr[@]}"</code>	<code>\$arr</code>
Longitud	<code>\${#arr[@]}</code>	<code>count \$arr</code>
Añadir	<code>arr+=("x")</code>	<code>set -a arr "x"</code>

Bucles

CONCEPTO	BASH / ZSH	FISH
For rango	<code>{1..5}</code> + do/done	<code>(seq 1 5)</code> + end
For lista	<code>"\${arr[@]}"</code> + do/done	<code>\$arr</code> + end
Cierre	<code>done</code>	<code>end</code>

06. Compatibilidad y Ecosistema

El problema POSIX

```
1 # Esto funciona en Bash/Zsh pero NO en Fish
2 export GOOGLE_CLOUD_PROJECT="mi-proyecto"
3 source /ruta/init.sh
```

```
1 # Equivalente en Fish
2 set -gx GOOGLE_CLOUD_PROJECT "mi-proyecto"
3 bass source /ruta/init.sh
```

Plugins Fisher esenciales

PLUGIN	FUNCIÓN	DESCRIPCIÓN
<code>edc/bass</code>	Ejecuta scripts Bash en Fish	Permite usar herramientas CLI diseñadas para Bash
<code>jorgebucaran/nvm.fish</code>	NVM nativo ultra-rápido	Gestión de versiones Node.js sin ralentizar el shell
<code>patrickf1/fzf.fish</code>	Buscador difuso (Ctrl+R visual)	Reemplazo moderno para <code>history</code> con vista previa de comandos
<code>jethrokuan/z</code>	Salto a carpetas frecuentes	Navegación rápida basada en uso reciente
<code>IlanCosman/tide</code>	Prompt asíncrono moderno	Prompt personalizable con información de Git, tiempo, etc.
<code>franciscolourenco/done</code>	Notificación de comandos largos	Notifica al terminar comandos que tardan más de 10 segundos

Instalación rápida de plugins

For Fish (Fisher)

```
1  fisher install edc/bass
2  fisher install patrickf1/fzf.fish
```

For Zsh (Oh My Zsh)

```
1  # En ~/.zshrc
2  plugins=(git fzf)
3  source $ZSH/oh-my-zsh.sh
```

fzf: Buscador difuso

```
src/util/eventbox_test.go
src/curses/curses_test.go
src/util/atomicbool.go
src/util/util_test.go
src/tokenizer_test.go
src/chunklist_test.go
src/algo/algo_test.go
src/util/eventbox.go
src/curses/curses.go
src/pattern_test.go
src/options_test.go
src/history_test.go
src/reader_test.go
src/merger_test.go
src/cache_test.go
src/util/util.go
src/tokenizer.go
src/item_test.go
src/constants.go
src/chunklist.go
src/ansi_test.go
src/algo/algo.go
src/terminal.go
src/fzf/main.go
src/pattern.go
> src/options.go
src/matcher.go
src/history.go
src/reader.go
src/merger.go
src/cache.go
src/item.go
src/core.go
src/ansi.go
35/63
```

```
> .go$
```

```
package fzf

import (
    "fmt"
    "os"
    "regexp"
    "strconv"
    "strings"
    "unicode/utf8"

    "github.com/junegunn/fzf/src/curses"
    "github.com/junegunn/go-shellwords"
)

const usage = `usage: fzf [options]

Search
-x, --extended      Extended-search mode
                     (enabled by default; +x or --no-extended to disable)
-e, --exact         Enable Exact-match
-i                 Case-insensitive match (default: smart-case match)
+i                 Case-sensitive match
-n, --nth=N[,...]  Comma-separated list of field index expressions
                     for limiting search scope. Each can be a non-zero
                     integer or a range expression ([BEGIN]..[END]).
                     Transform the presentation of each line using
                     field index expressions
--with-nth=N[,...] Transform the presentation of each line using
                     field index expressions
-d, --delimiter=STR Field delimiter regex (default: AWK-style)
+s, --no-sort       Do not sort the result
--tac              Reverse the order of the input
--tiebreak=CRI[,...] Comma-separated list of sort criteria to apply
                     when the scores are tied [length|begin|end|index]
                     (default: length)
```

eza: ls mejorado

```
src/git/test-git-repos on / main [$!?]  
> eza -l -git -git-repos --header  
Permissions Size User Date Modified Git Repo Name  
drwxr-xr-x - ces 30 Jul 08:30 -I - pr-1198 eza  
drwxr-xr-x - ces 30 Jul 08:30 -I | v6.9-rc3 linux  
drwxr-xr-x - ces 30 Jul 09:20 -I - master nixpkgs  
drwxr-xr-x - ces 30 Jul 08:42 -I | v1.15.0 starship
```

```
src/git/test-git-repos on / main [$!?]  
+> eza -l -git -T --hyperlink --header eza/src/output/  
Permissions Size User Date Modified Git Name  
drwxr-xr-x ces 30 Jul 08:30 -- eza/src/output  
-rw-r--r-- 8.5k ces 30 Jul 08:30 -- cell.rs  
-rw-r--r-- 19k ces 30 Jul 08:30 -- details.rs  
-rw-r--r-- 845 ces 30 Jul 08:30 -- escape.rs  
-rw-r--r-- 45k ces 30 Jul 08:30 -- file_name.rs  
-rw-r--r-- 2.0k ces 30 Jul 08:30 -- grid.rs  
-rw-r--r-- 10k ces 30 Jul 08:30 -- grid_details.rs  
-rw-r--r-- 17k ces 30 Jul 08:30 -- icons.rs  
-rw-r--r-- 36k ces 30 Jul 08:30 -- lines.rs  
-rw-r--r-- 1.4k ces 30 Jul 08:30 -- mod.rs  
drwxr-xr-x - ces 30 Jul 08:30 -- render  
-rw-r--r-- 1.2k ces 30 Jul 08:30 -- blocks.rs  
-rw-r--r-- 399 ces 30 Jul 08:30 -- filetype.rs  
-rw-r--r-- 4.0k ces 30 Jul 08:30 -- groups.rs  
-rw-r--r-- 3.7k ces 30 Jul 08:30 -- inodes.rs  
-rw-r--r-- 2.1k ces 30 Jul 08:30 -- links.rs  
-rw-r--r-- 770 ces 30 Jul 08:30 -- mod.rs  
-rw-r--r-- 4.2k ces 30 Jul 08:30 -- octal.rs  
-rw-r--r-- 9.2k ces 30 Jul 08:30 -- permissions.rs  
-rw-r--r-- 1.5k ces 30 Jul 08:30 -- securityctx.rs  
-rw-r--r-- 5.6k ces 30 Jul 08:30 -- sizes.rs  
-rw-r--r-- 73k ces 30 Jul 08:30 -- times.rs  
-rw-r--r-- 3.1k ces 30 Jul 08:30 -- users.rs  
-rw-r--r-- 17k ces 30 Jul 08:30 -- table.rs  
-rw-r--r-- 9.1k ces 30 Jul 08:30 -- time.rs  
-rw-r--r-- 9.5k ces 30 Jul 08:30 -- tree.rs
```

```
src/git/test-git-repos on / main [$!?]  
+> |
```

```
eza on / pr-1198 [$?] is v0.10.1 via v1.72.0-nightly via  
> eza -l -git -o  
-rw-r--r-- 3.7k ces 30 Jul 05:34 -- build.rs  
-rw-r--r-- 10k ces 30 Jul 06:40 -- Cargo.lock  
-rw-r--r-- 2.2k ces 30 Jul 07:10 -- Cargo.toml  
-rw-r--r-- 3.1k ces 30 Jul 07:14 -N changes.txt  
-rw-r--r-- 1.9k ces 30 Jul 06:34 -- completions  
-rw-r--r-- 5.3k ces 30 Jul 06:34 -- devtools  
-rw-r--r-- 5.3k ces 30 Jul 06:34 -- Flake.lock  
-rw-r--r-- 1.9k ces 30 Jul 06:34 -- Flake.nix  
-rw-r--r-- 2.6k ces 30 Jul 06:34 -- Justfile  
-rw-r--r-- 1.1k ces 28 Jul 10:20 -- LICENCE  
-rw-r--r-- 1.9k ces 30 Jul 06:34 -- man  
-rw-r--r-- 3.1k ces 30 Jul 07:14 -N old.txt  
-rw-r--r-- 3.9k ces 30 Jul 06:34 -- README.md  
-rw-r--r-- 54 ces 29 Jul 18:36 -- result -> /nix/store/a7y3753vpwg71jdyja6wis5yl8hvb9-eza-0.10.1  
-rw-r--r-- 12k ces 30 Jul 06:34 -- rust-toolchain.toml  
-rw-r--r-- 455k ces 29 Jul 17:17 -- screenshots.png  
-rw-r--r-- 1.9k ces 30 Jul 05:34 -- src  
-rw-r--r-- 1.9k ces 30 Jul 05:34 -- target  
-rw-r--r-- 117 ces 30 Jul 06:34 -- treefmt.nix  
-rw-r--r-- 6.1k ces 30 Jul 05:05 -- Vagrantfile  
-rw-r--r-- 1.9k ces 30 Jul 06:34 -- xtests
```

```
eza on / pr-1198 [$?] is v0.10.1 via v1.72.0-nightly via  
+> eza -l -grid -tree-style relative  
-rw-r--r-- 3.7k ces 3 hours build.rs  
-rw-r--r-- 10k ces 1 hour Cargo.lock  
-rw-r--r-- 2.2k ces 1 hour Cargo.toml  
-rw-r--r-- 3.1k ces 1 hour changes.txt  
drwxr-xr-x - ces 2 hours completions  
drwxr-xr-x - ces 2 hours devtools  
-rw-r--r-- 5.3k ces 2 hours Flake.lock  
-rw-r--r-- 1.9k ces 2 hours Flake.nix  
-rw-r--r-- 2.6k ces 2 hours Justfile  
-rw-r--r-- 1.1k ces 1 day LICENCE  
drwxr-xr-x - ces 2 hours man  
-rw-r--r-- 3.1k ces 1 hour old.txt  
-rw-r--r-- 9.1k ces 1 hour README.md  
-rw-r--r-- 54 ces 14 hours result  
-rw-r--r-- 12k ces 2 hours rust-toolchain.toml  
-rw-r--r-- 455k ces 15 hours screenshots.png  
drwxr-xr-x - ces 15 hours snap  
drwxr-xr-x - ces 3 hours src  
drwxr-xr-x - ces 3 hours target  
-rw-r--r-- 117 ces 2 hours treefmt.nix  
-rw-r--r-- 6.1k ces 3 hours Vagrantfile  
drwxr-xr-x - ces 2 hours xtests
```

```
eza on / pr-1198 [$?] is v0.10.1 via v1.72.0-nightly via  
+> eza  
build.rs completions Flake.lock LICENCE README.md screenshots.png target xtests  
Cargo.lock completions Flake.nix man result rust-toolchain.toml snap treefmt.nix  
Cargo.toml devtools Justfile old.txt rust-toolchain.toml src Vagrantfile
```

```
eza on / pr-1198 [$?] is v0.10.1 via v1.72.0-nightly via  
+> |
```

Ejemplos del Día a Día (Fish)

gcpush

```
1  function gcpush
2      git add .
3      if test (count $argv) -gt 0
4          git commit -m "$argv[1]"
5      else
6          git commit -m (date "+%d %^b %y %H:%M")
7      end
8      git push
9  end
10
11  funcsave gcpush
```

Uso

```
1  $>gcpush "Actualización de README"
2  $>git log --oneline
3  14538a (HEAD -> main, origin/main, origin/HEAD) Actualización de README
```

O sin mensaje:

```
1  $>gcpush
2  $>git log --oneline
3  14538a (HEAD -> main, origin/main, origin/HEAD) 12 MAY 26 13:17
```

pulleverything

```
1  function pulleverything
2      for dir in */.git
3          echo ""
4          set_color --bold --underline yellow
5          echo (dirname $dir)
6          set_color normal
7          git -C (dirname $dir) pull
8      end
9  end
10
11  funcsave pulleverything
```

ft_printf

Already up to date.

repo_not_sync

Updating 42c9b8a..d1e2f3a

Fast-forward

README.md | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)

memo

```
1  function memo
2      set -l memo_file "$FISH_TOOLS_ROOT/memo.md"
3      if test (count $argv) -lt 2
4          echo "Usage: memo \"Description\" \"Command\""
5          return
6      end
7      set -l description $argv[1]
8      set -l command_text $argv[2]
9      if not test -f "$memo_file"
10         echo "# Personal Command Memo" > "$memo_file"
11     end
12     echo -e "\n## $description\n\\`\\`\\`bash\\`\\`\\`$command_text\n\\`\\`\\`" >> "$memo_file"
13     echo "✅ Saved to: $memo_file"
14 end
15
16 funcsave memo
```

Uso

```
1  memo "Titulo de la entrada o comando" "ls -l"
```

memoshow

```
1 function memoshow
2   cat "$FISH_TOOLS_ROOT/memo.md"
3 end
4
5 funcsave memoshow
```

Uso

```
1 $>memoshow
2 # Titulo de la entrada o comando
3 `ls -l`
4
5 # Comando anterior
6 `cat README.md`
7
8 # Comando de hace 1 mes
9 `git log --oneline`
```

statussearch

Estado Git de múltiples repositorios:

```
1  function statussearch
2      set -l repos (find . -name ".git" -type d)
3      for repo in $repos
4          set -l parent (dirname $repo)
5          echo -n "Checking $parent... "
6          git -C $parent fetch --quiet &
7          wait
8          echo -ne "\r\033[K"
9          set -l dinfo (git -C $parent status --porcelain)
10         set -l branch_info (git -C $parent status -sb)
11         if test -n "$dinfo"
12             set_color --bold yellow
13             echo "🔴 $parent (Cambios locales)"
14             set_color normal
15             git -C $parent status -s
16             echo ""
17         else if string match -q "**behind*" "$branch_info"
18             set_color --bold magenta
19             echo "🔴 $parent (Pendiente de PULL - Hay cambios en GitHub)"
20             set_color normal
21             echo ""
22         else if string match -q "**ahead*" "$branch_info"
23             set_color --bold cyan
24             echo "🟢 $parent (Pendiente de PUSH)"
25             set_color normal
26             echo ""
27         else
28             set_color grey
29             echo "✓ $parent"
30             set_color normal
31         end
32     end
33 end
```

```
1  $>find . -name ".git" -type d
2  ./repo1/.git
3  ./repo2/.git
4  ./repo3/.git
```

- `fetch --quiet` para no saturar la salida
- `status --porcelain` para detectar cambios locales
- `status -sb` para ver si estamos behind/ahead del remoto.

statussearch: Salida de ejemplo

✓ ./path/to/repo1-dev

🔧 ./path/to/repo2 (Cambios locales)

M project/src/test.c

📦 ./path/to/repo3/README_tutorial (Pendiente de PULL)

list_declarations

Escanea archivos C extrayendo firmas de funciones:

```
1  function list_declarations
2      if test (count $argv) -eq 0
3          echo "Usage: list_declarations file1.c file2.c ..."; return 1
4      end
5
6      for file in $argv
7          set -l regex '^' # Inicio de línea
8          set regex "$regex" "[ \t]*(static[ \t]+)?" # Opcional "static" al inicio
9          set regex "$regex" "[a-zA-Z_][a-zA-Z0-9_]*" # Tipo de retorno (int, void, char*, etc.)
10         set regex "$regex" "([ \t]|\\*)" # Espacios o asteriscos entre tipo y nombre
11         set regex "$regex" "[a-zA-Z_][a-zA-Z0-9_]*" # Nombre de la función
12         set regex "$regex" "[ \t]*\\" # Espacios opcionales antes del paréntesis de apertura
13
14         # Ejecutamos pasando la regex limpia
15         set -l output (command awk "/$regex/ { print \$0 \";\\" }" $file | tr -d '\r')
16
17         echo "// $file N:$(count $output)
18         string join \n $output
19     end
20 end
```

USO

```
1  $>list_declarations parser.c
2  // parser.c N:5
3  int ft_isdigit(int c);
4  void skip_spaces(const char *nptr, int *i);
5  void parse_sign(const char *str, int *i, int *sign);
6  int ft_atoi_strict(const char *str, int *out);
7  int check_args(int argc, char **argv);
```

PREGUNTA: ¿PODEMOS GENERALIZAR PARA TODO UN PROYECTO?

```
1  src/
2  |— main.c
3  |— parser.c
4  |— utils.c
```

```
1  function list_decl_bulk
2      for i in *.c
3          list_declarations $i
4      end
5  end
```

PREGUNTA: ¿PODEMOS GENERALIZAR PARA TODO UN PROYECTO?

```
1  src/
2  |— main.c
3  |— parser/
4  |   |— parser.c
5  |   |— parser_utils.c
6  |   |— lvalidation.c
7  |— rendering/
```

```
1  function list_decl_bulk
2    for i in **/*.c
3      list_declarations $i
4    end
5  end
```

```
1  function list_decl_bulk
2    for i in (find . -name "*.c")
3      list_declarations $i
4    end
5  end
```

```
1  function list_decl_bulk
2    find . -name "*.c" -exec list_declarations {} \;
3  end
```

08. Fish Tools Loader

Organización modular

Todas las funciones anteriores se agrupan en un repositorio estructurado:

```
1 fish_tools/
2 |─ init.fish      # Motor de carga
3 |─ groups/
4 |   |─ 42tools/   # list_declarations.fish
5 |   |─ gt/        # gcpush, pulleverything, statussearch
6 |   |─ utils/     # memo, memoshow
```

```
1 # init.fish - Motor de carga automática
2 set -gx FISH_TOOLS_ROOT (dirname (status filename))
3
4 for group_dir in $FISH_TOOLS_ROOT/groups/*/
5     for func_file in $group_dir*.fish
6         source $func_file
7     end
8 end
9
10 echo "fish_tools: " (count $FISH_TOOLS_ROOT/groups/*/*.fish) "funciones cargadas"
```

Sincronización vía Git

```
1  # En ~/.config/fish/config.fish
2  set -gx FISH_TOOLS_ROOT ~/path/to/fish_tools
3  source $FISH_TOOLS_ROOT/init.fish
```

Ventajas:

- Las funciones se cargan automáticamente por grupos temáticos
- El autocompletado con TAB funciona para todos los subcomandos
- Se sincroniza entre máquinas con `git pull` si se guarda en un repositorio
- Cada función es un archivo independiente → fácil de mantener

09. Conclusión

¿Cuál elegir?

CONTEXTO	RECOMENDACIÓN
Servidores / CI/CD / Docker	Bash
Compartir herramientas en equipo	Bash
macOS (sin configurar nada)	Zsh
Personalización total	Zsh + Oh My Zsh
Productividad local diaria	Fish
"Que funcione desde el minuto uno"	Fish

¡Gracias!
mlermo-j

42 Málaga · Shells